

Algoritmos

2026-04-01

R Markdown

This is an R Markdown document. Markdown is a simple formatting syntax for authoring HTML, PDF, and MS Word documents. For more details on using R Markdown see <http://rmarkdown.rstudio.com>.

When you click the **Knit** button a document will be generated that includes both content as well as the output of any embedded R code chunks within the document. You can embed an R code chunk like this:

Primeros comandos de R

R es un lenguaje muy parecido a Matlab y trabaja fundamentalmente con variables que en general pueden ser matrices.

```
A=38
A=40
B=60
C=B-A
C
```

```
## [1] 20
```

```
Y=40
alpha<-30
```

Uso de datasets o base de datos internos de R

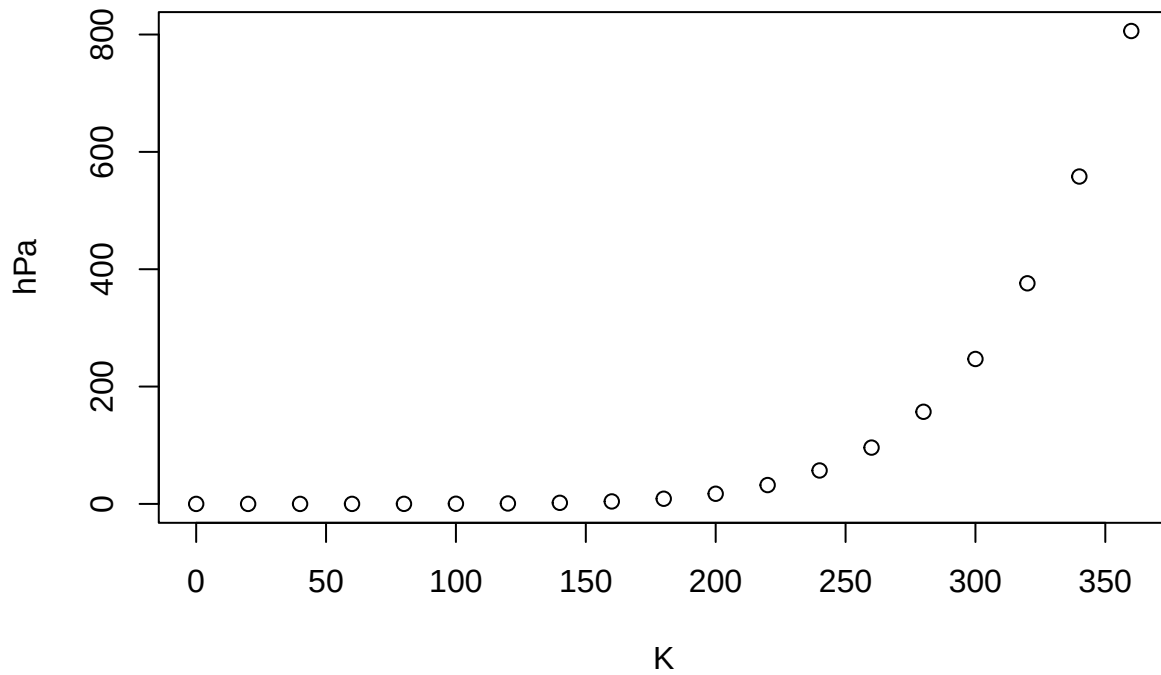
Usando el comando `data` en la consola obtenemos distintos datos ya cargados en la base de datos R, estos datos están dados en tablas de las cuales podemos elegir columnas específicas si escribimos signo `$` después del comando de información específico y elegimos la columna de datos.

Including Plots

You can also embed plots, for example:

```
plot(pressure, main="Presión del gas ideal",ylab="hPa",xlab="K")
```

Presión del gas ideal



Comando Plot

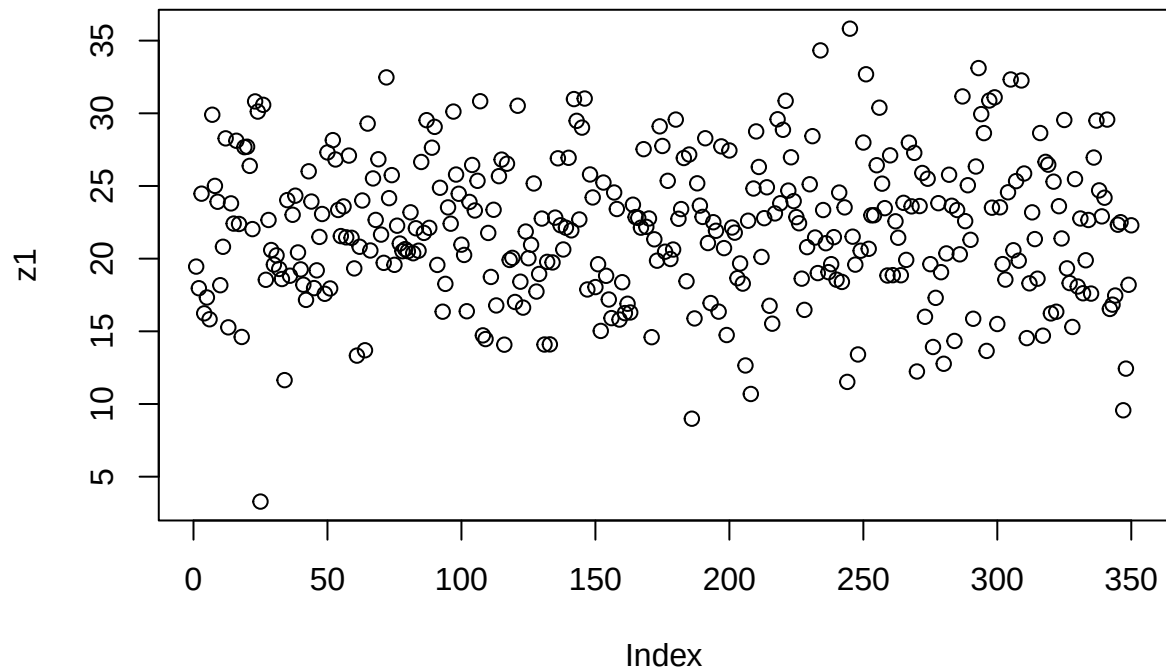
Es un comando que permite graficar cualquier fuente de datos que le asignemos, con `main` ponemos el título, con `ylab` ponemos título al eje ordenadas y con `xlab` ponemos título al eje abscisas. En caso de dudas de como utilizar el comando podemos escribir `?plot` en la consola, lo cual abrirá una ventana que explicará en detalle el comando.

Note that the `echo = FALSE` parameter was added to the code chunk to prevent printing of the R code that generated the plot.

funciones estadísticas

```
z1<-rnorm(350,22,5)
```

```
plot(z1)
```



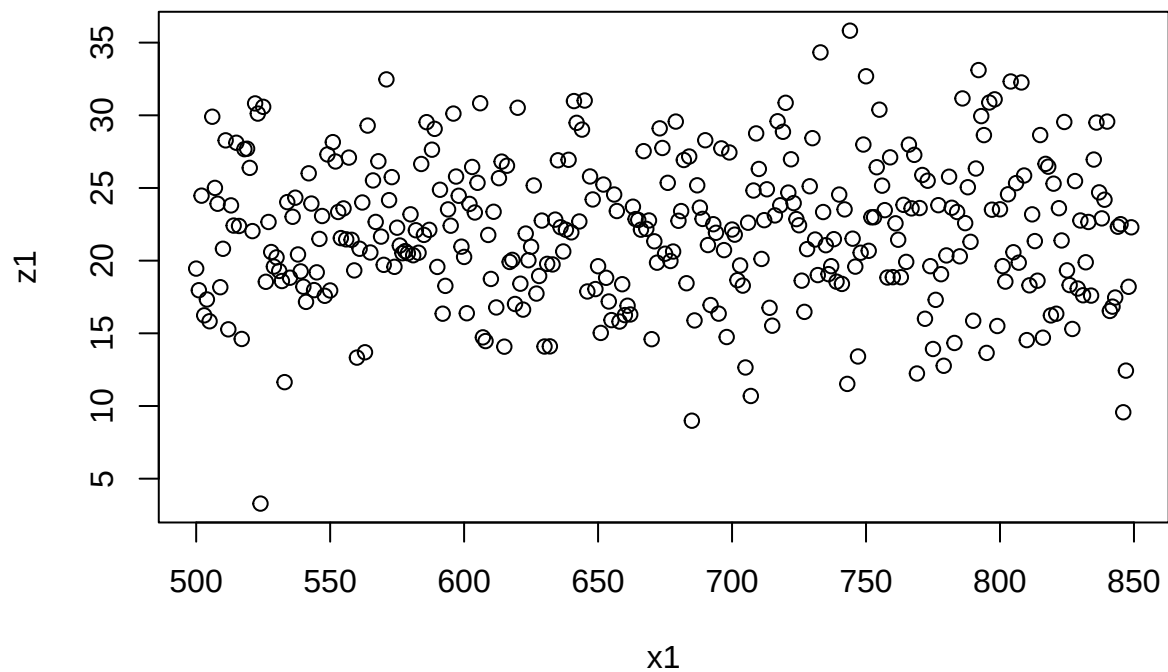
```
w1<-length(z1)
w1
```

```
## [1] 350
```

```
x1<-(500:849)
x1
```

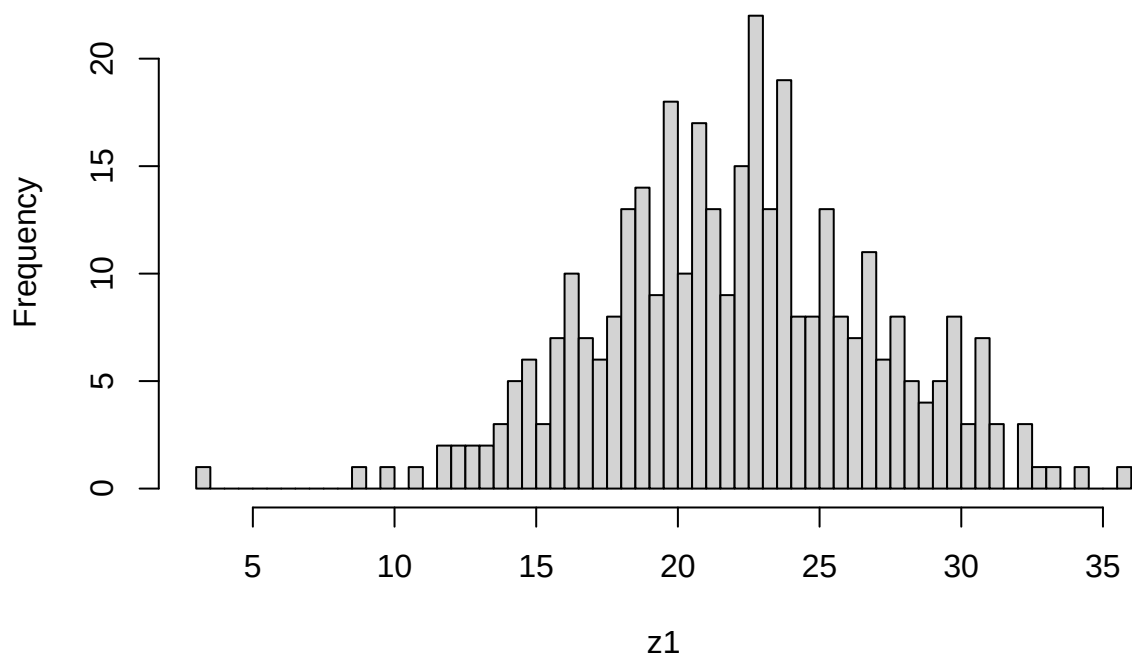
```
## [1] 500 501 502 503 504 505 506 507 508 509 510 511 512 513 514 515 516 517
## [19] 518 519 520 521 522 523 524 525 526 527 528 529 530 531 532 533 534 535
## [37] 536 537 538 539 540 541 542 543 544 545 546 547 548 549 550 551 552 553
## [55] 554 555 556 557 558 559 560 561 562 563 564 565 566 567 568 569 570 571
## [73] 572 573 574 575 576 577 578 579 580 581 582 583 584 585 586 587 588 589
## [91] 590 591 592 593 594 595 596 597 598 599 600 601 602 603 604 605 606 607
## [109] 608 609 610 611 612 613 614 615 616 617 618 619 620 621 622 623 624 625
## [127] 626 627 628 629 630 631 632 633 634 635 636 637 638 639 640 641 642 643
## [145] 644 645 646 647 648 649 650 651 652 653 654 655 656 657 658 659 660 661
## [163] 662 663 664 665 666 667 668 669 670 671 672 673 674 675 676 677 678 679
## [181] 680 681 682 683 684 685 686 687 688 689 690 691 692 693 694 695 696 697
## [199] 698 699 700 701 702 703 704 705 706 707 708 709 710 711 712 713 714 715
## [217] 716 717 718 719 720 721 722 723 724 725 726 727 728 729 730 731 732 733
## [235] 734 735 736 737 738 739 740 741 742 743 744 745 746 747 748 749 750 751
## [253] 752 753 754 755 756 757 758 759 760 761 762 763 764 765 766 767 768 769
## [271] 770 771 772 773 774 775 776 777 778 779 780 781 782 783 784 785 786 787
## [289] 788 789 790 791 792 793 794 795 796 797 798 799 800 801 802 803 804 805
## [307] 806 807 808 809 810 811 812 813 814 815 816 817 818 819 820 821 822 823
## [325] 824 825 826 827 828 829 830 831 832 833 834 835 836 837 838 839 840 841
## [343] 842 843 844 845 846 847 848 849
```

```
plot(x1,z1)
```



```
hist(z1, main = "histograma de edades", breaks = 50)
```

histograma de edades



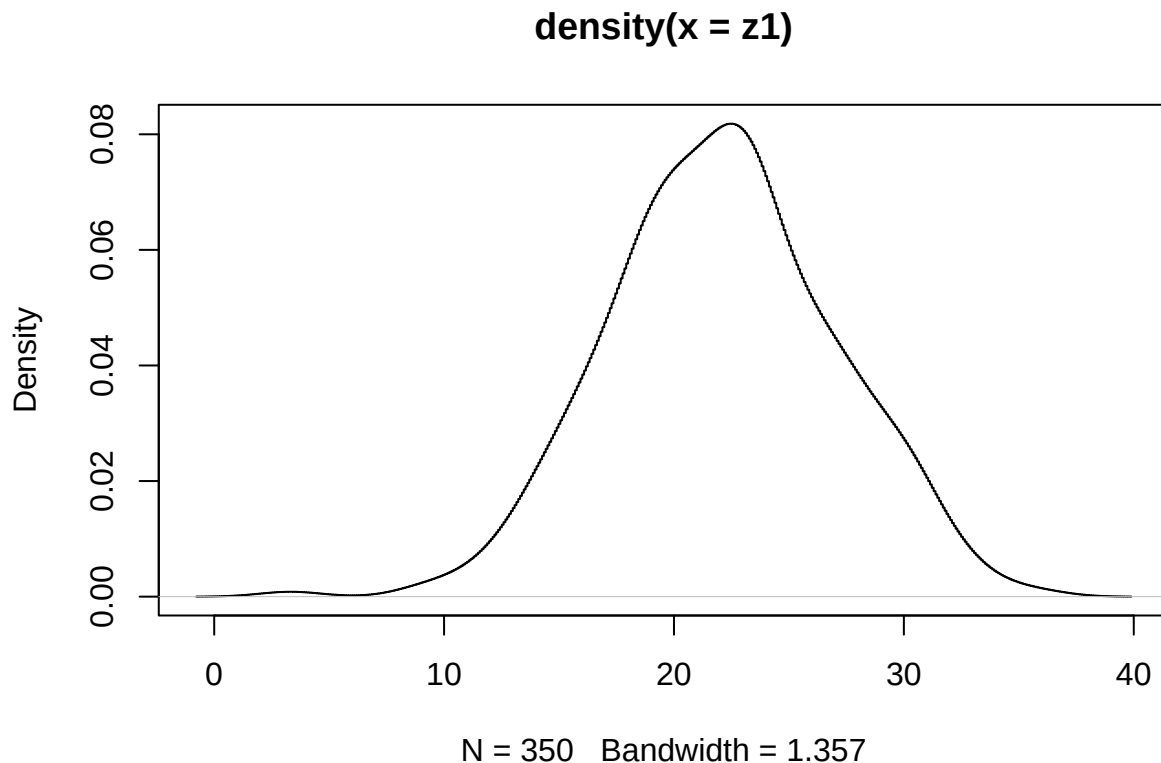
```
density(z1,type="d")
```

```
## Warning: In density.default(z1, type = "d") :  
##   extra argument 'type' will be disregarded
```

```
##  
## Call:
```

```
## density.default(x = z1, type = "d")
##
## Data: z1 (350 obs.); Bandwidth 'bw' = 1.357
##
##      x              y
## Min.   :-0.7842   Min.   :9.391e-06
## 1st Qu.: 9.3851   1st Qu.:7.732e-04
## Median :19.5543   Median :1.035e-02
## Mean   :19.5543   Mean    :2.454e-02
## 3rd Qu.:29.7236   3rd Qu.:4.510e-02
## Max.   :39.8928   Max.    :8.184e-02
```

```
plot(density(z1),type="s")
```



Ejercicio 1 de Algoritmos

Consigna: Las últimas tres cifras de mi DNI son 919 Crear una variable que tenga ese número

```
DNI=919
```

Consigna: crear un vector que tenga todos los números enteros desde el 1 al 919

```
for(i in 1:5) print(1:i)
```

```
## [1] 1
## [1] 1 2
## [1] 1 2 3
## [1] 1 2 3 4
## [1] 1 2 3 4 5
```

```
secuencia_dni<- (1:919)
secuencia_dni
```

```

## [1] 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18
## [19] 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36
## [37] 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54
## [55] 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72
## [73] 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90
## [91] 91 92 93 94 95 96 97 98 99 100 101 102 103 104 105 106 107 108
## [109] 109 110 111 112 113 114 115 116 117 118 119 120 121 122 123 124 125 126
## [127] 127 128 129 130 131 132 133 134 135 136 137 138 139 140 141 142 143 144
## [145] 145 146 147 148 149 150 151 152 153 154 155 156 157 158 159 160 161 162
## [163] 163 164 165 166 167 168 169 170 171 172 173 174 175 176 177 178 179 180
## [181] 181 182 183 184 185 186 187 188 189 190 191 192 193 194 195 196 197 198
## [199] 199 200 201 202 203 204 205 206 207 208 209 210 211 212 213 214 215 216
## [217] 217 218 219 220 221 222 223 224 225 226 227 228 229 230 231 232 233 234
## [235] 235 236 237 238 239 240 241 242 243 244 245 246 247 248 249 250 251 252
## [253] 253 254 255 256 257 258 259 260 261 262 263 264 265 266 267 268 269 270
## [271] 271 272 273 274 275 276 277 278 279 280 281 282 283 284 285 286 287 288
## [289] 289 290 291 292 293 294 295 296 297 298 299 300 301 302 303 304 305 306
## [307] 307 308 309 310 311 312 313 314 315 316 317 318 319 320 321 322 323 324
## [325] 325 326 327 328 329 330 331 332 333 334 335 336 337 338 339 340 341 342
## [343] 343 344 345 346 347 348 349 350 351 352 353 354 355 356 357 358 359 360
## [361] 361 362 363 364 365 366 367 368 369 370 371 372 373 374 375 376 377 378
## [379] 379 380 381 382 383 384 385 386 387 388 389 390 391 392 393 394 395 396
## [397] 397 398 399 400 401 402 403 404 405 406 407 408 409 410 411 412 413 414
## [415] 415 416 417 418 419 420 421 422 423 424 425 426 427 428 429 430 431 432
## [433] 433 434 435 436 437 438 439 440 441 442 443 444 445 446 447 448 449 450
## [451] 451 452 453 454 455 456 457 458 459 460 461 462 463 464 465 466 467 468
## [469] 469 470 471 472 473 474 475 476 477 478 479 480 481 482 483 484 485 486
## [487] 487 488 489 490 491 492 493 494 495 496 497 498 499 500 501 502 503 504
## [505] 505 506 507 508 509 510 511 512 513 514 515 516 517 518 519 520 521 522
## [523] 523 524 525 526 527 528 529 530 531 532 533 534 535 536 537 538 539 540
## [541] 541 542 543 544 545 546 547 548 549 550 551 552 553 554 555 556 557 558
## [559] 559 560 561 562 563 564 565 566 567 568 569 570 571 572 573 574 575 576
## [577] 577 578 579 580 581 582 583 584 585 586 587 588 589 590 591 592 593 594
## [595] 595 596 597 598 599 600 601 602 603 604 605 606 607 608 609 610 611 612
## [613] 613 614 615 616 617 618 619 620 621 622 623 624 625 626 627 628 629 630
## [631] 631 632 633 634 635 636 637 638 639 640 641 642 643 644 645 646 647 648
## [649] 649 650 651 652 653 654 655 656 657 658 659 660 661 662 663 664 665 666
## [667] 667 668 669 670 671 672 673 674 675 676 677 678 679 680 681 682 683 684
## [685] 685 686 687 688 689 690 691 692 693 694 695 696 697 698 699 700 701 702
## [703] 703 704 705 706 707 708 709 710 711 712 713 714 715 716 717 718 719 720
## [721] 721 722 723 724 725 726 727 728 729 730 731 732 733 734 735 736 737 738
## [739] 739 740 741 742 743 744 745 746 747 748 749 750 751 752 753 754 755 756
## [757] 757 758 759 760 761 762 763 764 765 766 767 768 769 770 771 772 773 774
## [775] 775 776 777 778 779 780 781 782 783 784 785 786 787 788 789 790 791 792
## [793] 793 794 795 796 797 798 799 800 801 802 803 804 805 806 807 808 809 810
## [811] 811 812 813 814 815 816 817 818 819 820 821 822 823 824 825 826 827 828
## [829] 829 830 831 832 833 834 835 836 837 838 839 840 841 842 843 844 845 846
## [847] 847 848 849 850 851 852 853 854 855 856 857 858 859 860 861 862 863 864
## [865] 865 866 867 868 869 870 871 872 873 874 875 876 877 878 879 880 881 882
## [883] 883 884 885 886 887 888 889 890 891 892 893 894 895 896 897 898 899 900
## [901] 901 902 903 904 905 906 907 908 909 910 911 912 913 914 915 916 917 918
## [919] 919

```

Ejercicio 2

Calcular la suma de todos los valores del vector `Secuencia_dni`

```
Total<-0
Valor_final<-length(secuencia_dni)
for (i in 1:Valor_final)
Total<-Total + i
Total
```

```
## [1] 422740
```

Ejercicio 3

Repetir el ejercicio anterior en Python

```
suma = 0

for i in range(1,920):
    suma+=i

print(suma)
```

```
## 422740
```

Ejercicio 4

¿Cuánto tarda en correr el código del ejercicio 2?

```
Sys.time()
```

```
## [1] "2026-06-03 22:19:15 UTC"
```

```
(for(i in 1:100){})
```

```
## NULL
```

```
Sys.time
```

```
## function ()
## .POSIXct(.Internal(Sys.time()))
## <bytecode: 0x567962866e40>
## <environment: namespace:base>
```

```
inicio<-Sys.time()
Total<-0
Valor_final<-1000000*length(secuencia_dni)
for (i in 1:Valor_final){
Total<-Total + i
}
final<-Sys.time()
final-inicio
```

```
## Time difference of 15.25685 secs
```

Ejercicio 5

Repetir el ejercicio anterior utilizando la cabeza de Gauss

Ejercicio 6

Generar una serie de Fibonacci

```
n <- 32
fib <- numeric(n)
fib[1] <- 0
fib[2] <- 1

for (i in 3:n) {
  fib[i] <- fib[i-1] + fib[i-2]
}

print(fib)
```

```
## [1]      0      1      1      2      3      5      8     13     21
## [10]     34     55     89    144    233    377    610    987   1597
## [19]    2584    4181    6765   10946   17711   28657   46368   75025  121393
## [28]  196418  317811  514229  832040 1346269
```

Ejercicio 7.1

Suma de serie Fibonacci

```
n <- 32

fib <- numeric(n)
fib[1] <- 0
fib[2] <- 1

# Variable para acumular la suma
suma_fibonacci <- fib[1] + fib[2]

for (i in 3:n) {
  fib[i] <- fib[i-1] + fib[i-2]
  suma_fibonacci <- suma_fibonacci + fib[i]
}

print(paste("La serie es:", paste(fib, collapse = ", ")))
```

```
## [1] "La serie es: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, 610, 987, 1597, 2584, 4181"
print(paste("La suma total es:", suma_fibonacci))
```

```
## [1] "La suma total es: 3524577"
```

Ejercicio 7.2

Productoria de los últimos 3 números del DNI. Es factorial, lo más grande que aguanta es el 69 factorial, luego se traba el código.

No lo podemos hacer porque se traba.

Ejercicio 8

Definición matemática recurrente ¿Cuántas iteraciones se necesitan para generar un número de la serie mayor que 1.000.000?


```
f0 <- 0
f1 <- 1
iteraciones <- 1

while (f1 <= 1000000) {
  temp <- f0 + f1
  f0 <- f1
  f1 <- temp
  iteraciones <- iteraciones + 1
}

print(paste("Iteraciones necesarias para generar un número mayora a 1000000 es:", iteraciones + 1))

## [1] "Iteraciones necesarias para generar un número mayora a 1000000 es: 32"
```

Ejercicio 9

Ordenación de un vector por el método burbuja

```
#Tomo una muestra de 10 números entre 1 y 25
x<-sample(1:600,10)

# Definimos una función BURBUJA que recibe un paquete de datos x para ordenar
burbuja <- function(x) {
  n <- length(x)
  for(j in 1:(n-1)) {
    for(i in 1:(n-j)) {
      if(x[i] > x[i+1]) {
        temp <- x[i]
        x[i] <- x[i+1]
        x[i+1] <- temp
      }
    }
  }
  return(x)
}
res<-burbuja(x)

#Muestra obtenida
x

## [1] 165 415 268 116 503 440 336 329 429 302

# Muestra ordenada
res

## [1] 116 165 268 302 329 336 415 429 440 503

# Ordenación con el comando SORT de R-Cran
sort(x)

## [1] 116 165 268 302 329 336 415 429 440 503

# INSTALAMOS los paquetes
# install.packages("microbenchmark")
# install.packages("ggplot2")
```

```

library(microbenchmark)
library(ggplot2)

# EJECUTAMOS EL BENCHMARK
# Repetimos 10 veces cada uno para tener una muestra estadística
comparativa <- microbenchmark(
  Metodo_Burbuja = burbuja(x),
  Sort_Nativo = sort(x),
  times = 10
)

# Imprimir resumen en consola
print(comparativa)

## Unit: microseconds
##          expr   min    lq   mean median    uq   max neval
## Metodo_Burbuja 6.59  7.05  9.839  7.965  9.75 26.17    10
## Sort_Nativo    25.44 26.49 41.315 27.635 33.96 147.66    10

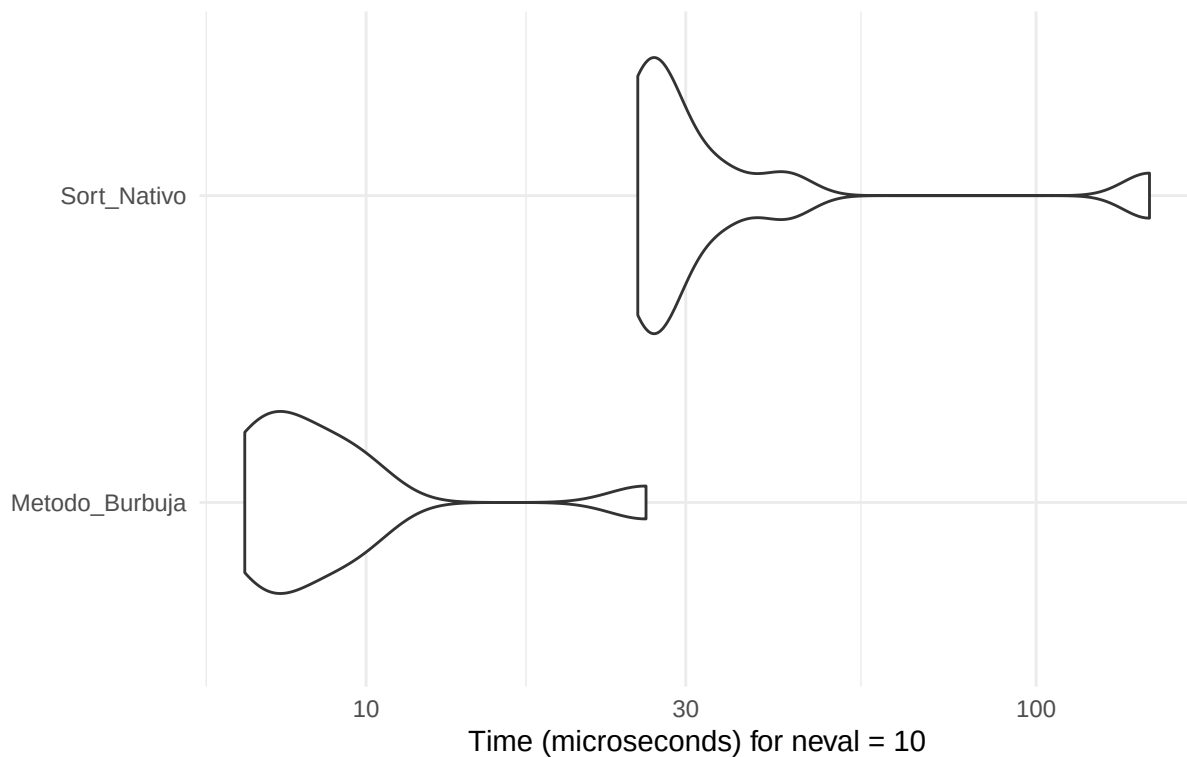
# Gráfico de Violín para investigar el comportamiento del CPU
autoplot(comparativa) +
  labs(title = "Comparativa de Performance: Burbuja vs Sort",
        subtitle = "La escala vertical es logarítmica debido a la diferencia abismal",
        y = "Tiempo de respuesta",
        x = "Algoritmo") +
  theme_minimal()

## Warning: `aes_string()` was deprecated in ggplot2 3.0.0.
## i Please use tidy evaluation idioms with `aes()`.
## i See also `vignette("ggplot2-in-packages")` for more information.
## i The deprecated feature was likely used in the microbenchmark package.
## Please report the issue at
## <https://github.com/joshuaulrich/microbenchmark/issues/>.
## This warning is displayed once per session.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.

```

Comparativa de Performance: Burbuja vs Sort

La escala vertical es logarítmica debido a la diferencia abismal



#El "violín" de la Burbuja estará muy por encima del de Sort. Esto confirma que la computadora realiza

Ejercicion 10

La penitencia de Newton. Desarrollar dos algoritmos que hagan este trabajo, por ejemplo para sumar desde 1 hasta $1 * 10^{10}$ y verificar cuál de los dos es más eficiente.

```
# Definimos el límite: 10^10 (diez mil millones)
# Nota: Usamos opciones para evitar la notación científica y asegurar precisión
options(scipen = 999)
n <- 1e10

# --- ALGORITMO 1: Fuerza Bruta (Iterativo) ---
# Advertencia: Para 10^10 este método tardará mucho en R.
# Probaremos con un número más pequeño para el ejemplo iterativo (10^7)
sumar_iterativo <- function(num) {
  suma <- 0
  for (i in 1:num) {
    suma <- suma + i
  }
  return(suma)
}

# --- ALGORITMO 2: Fórmula Matemática (Constante) ---
sumar_formula <- function(num) {
  # Usamos la fórmula de Gauss: n * (n + 1) / 2
  # En R, para números tan grandes, trabajamos con aritmética de precisión
```

```

    return( (num * (num + 1)) / 2 )
}

# --- PRUEBA DE EFICIENCIA ---

# Medir Algoritmo Matemático (Instantáneo incluso para 1010)
inicio_f <- Sys.time()
resultado_f <- sumar_formula(n)
fin_f <- Sys.time()

cat("Resultado (Fórmula):", resultado_f, "\n")

## Resultado (Fórmula): 50000000005000003584
cat("Tiempo (Fórmula):", fin_f - inicio_f, "segundos\n\n")

## Tiempo (Fórmula): 0.0009782314 segundos

# Medir Algoritmo Iterativo (Solo con 107 para no bloquear la PC)
n_pequeno <- 1e7
inicio_i <- Sys.time()
resultado_i <- sumar_iterativo(n_pequeno)
fin_i <- Sys.time()

cat("Resultado Iterativo (para 107):", resultado_i, "\n")

## Resultado Iterativo (para 107): 50000005000000
cat("Tiempo Iterativo (para 107):", fin_i - inicio_i, "segundos\n")

## Tiempo Iterativo (para 107): 0.1622896 segundos

```

Ejercicio 11

Medir la performance del método k-mean de agrupamiento. Realizar una introducción teórica antes de medir con gráfico de violin.

Resuelto en otro archivo